AI Agent Platform

Architecture Design and Technical Documentation

Contents

Chapter 1

Architecture Design

This chapter describes what the platform does, how it is put together, and how a request travels through it. It also records the design choices that shaped the system and what those choices cost.

1.1 What the platform is, and who uses it

The AI Agent Platform lets a person build reusable agents. An agent is an LLM with a fixed configuration: a name, an icon, a description, a chosen model, a system prompt, and a list of tools it may use. An agent can be run in two ways. A person can chat with it, or a schedule can run it automatically.

Every run is recorded, no matter how it started. The record holds the request, the answer, every tool the model called with its arguments and results, how long the run took, and how it ended. This record is the second product of the platform, alongside the chat itself. A person can always go back and see what an agent did.

Two different things can start a run, and the design treats that difference as a real one rather than an accident of implementation:

- **A person**, working in the browser. They create and edit agents, chat with them, manage chat threads, manage triggers, download run logs, and maintain the knowledge documents their agents can search.
- **The trigger scheduler**, a clock inside the service itself. It has no browser session and no user behind it. It starts an agent when a schedule comes due.

Two outside systems take part in a run but can never start one: the LLM provider that produces the answer, and whatever web endpoint a tool is pointed at. They appear as secondary actors because a run cannot finish without them.

Figure 1 shows the actors, what each of them can do, and the fact that chat and scheduled triggers both end in a recorded run.

Three things in that picture are worth stating directly.

- **Configuring an agent is not separate from managing it.** The system prompt and the tool list are fields on the agent, saved by the same write that saves its name or its status. There is no separate configuration screen and no separate endpoint, so an agent's identity and its behaviour cannot drift apart.
- **The three ways to start an agent differ only in what starts them.** Chatting, firing a trigger by hand, and firing a trigger on schedule all include the same step: execute an agent run. They use the same agent settings, the same tools, and the same model call. Modelling that step once is what stops the platform from growing three slightly different execution paths. Section 4 follows it end to end.
- **Every execution is recorded, whether or not anyone looks.** Executing a run always includes recording it. Reading run history is therefore a plain action a person chooses to take, not a step inside execution.

Implementation. The single human actor covers every route the frontend exposes: agent management, chat, triggers, run history, and the knowledge library. The scheduler actor is a background loop with no HTTP request and no session behind it (section 4.2). This proof of concept has no login and no roles, so there is exactly one kind of human actor to model.

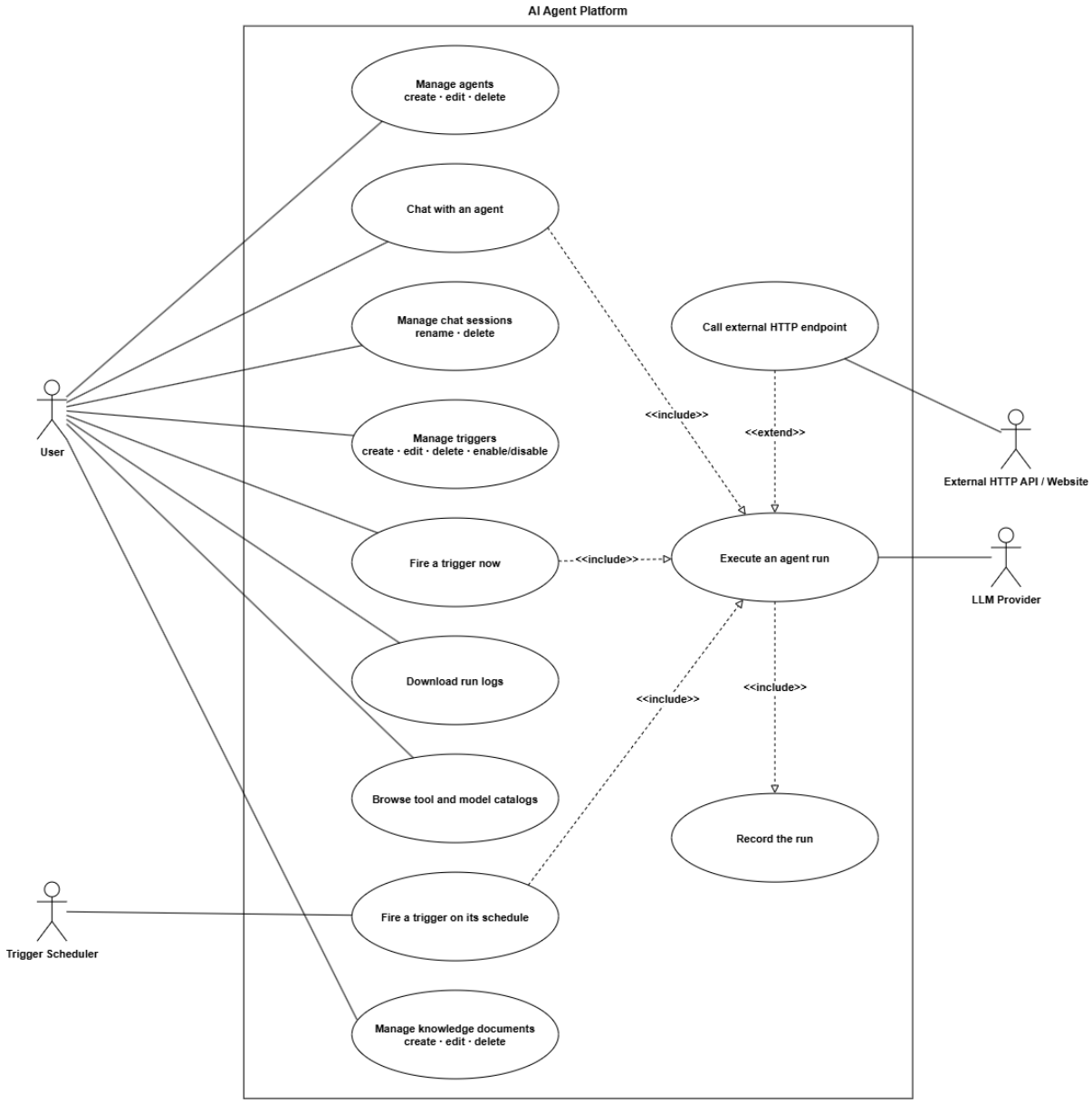

Figure 1: AI Agent Platform use cases and actor interactions

1.2 Architecture

The system has two parts: a browser application the user works in, and a backend that stores agents, runs them, and keeps history. The backend is split into components. Each component has one job and reaches the others through a defined interface.

Those interfaces are called ports. A component asks for an LLM or for storage through a port and receives whichever implementation is configured. The same component therefore works against an in-memory store and a mock model during development, and against PostgreSQL and Gemini in a real deployment, without any change to its own code.

Figure 2 shows how the browser and the scheduler reach the backend components, and how those components depend on the provider and persistence ports.

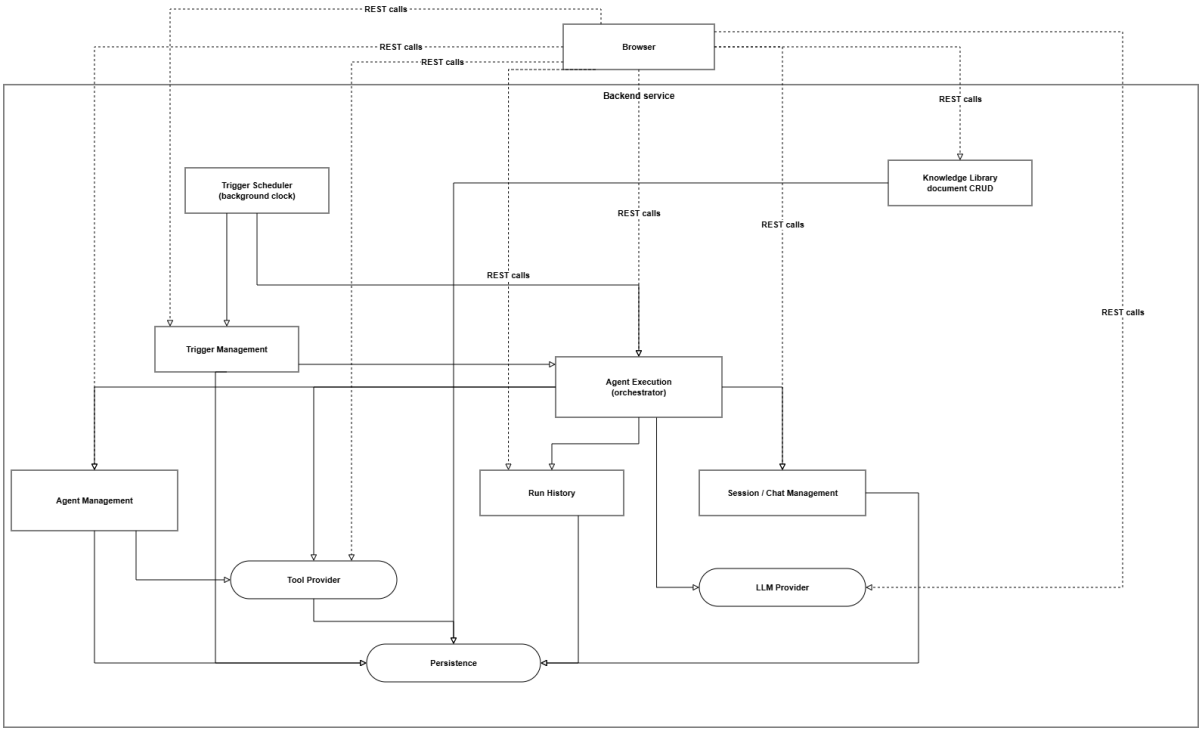

Figure 2: Backend architecture

Agent Execution is the orchestrator. It touches more of the system than anything else: agent settings, tools, the LLM, run history, and sessions. Coordinating those for one agent run is its job. Every other component depends only on the few services it actually needs.

Two ports have more than one implementation, chosen by an environment setting:

Port	Implementations	Selected by
LLM Provider	Mock model, or Gemini through Google ADK	`LLM_PROVIDER=mock|adk_gemini`
Storage	In-memory store or PostgreSQL	`STORE_BACKEND=memory|postgres`
Tool Provider	Four built-in tools	Fixed for this deliverable

Implementation. The backend is one FastAPI service, organised as modules under `app/modules/`: `agents` (Agent Management), `execution` (Agent Execution), `llm` (LLM Provider), `tools` (Tool Provider), `knowledge` (Knowledge Library), `runs` (Run History), `sessions` (Session Management), and `triggers` (Trigger Management and the scheduler). Each port is a Python `Protocol`. The implementation behind it is chosen in exactly one file, `app/container.py`, from

environment configuration. It is never wired case by case inside a service. The frontend is a React single-page application whose only channel to the backend is `fetch` calls to `/api/*`, all of which go through one client module, so no component or hook talks to the network directly.

1.3 What each component is responsible for

Each component below is described by what it does, why it exists as its own component, how it connects to the rest, and where it lives in the code.

1.3.1 Frontend

The browser application is what the user sees. It provides pages for managing agents, chatting, managing triggers, viewing run history, and maintaining knowledge documents.

Every user action becomes an API request. All API calls are collected in one module, so the rest of the frontend does not need to know the URL shapes, the error format, or the streaming protocol.

The frontend never calls the LLM or a tool itself. It sends a request to the backend, and the backend decides what happens and stores the result.

Implementation. `AgentPlatform-FrontEnd/client`, with `src/lib/api-client.ts` as the only module that calls `fetch`.

1.3.2 Agent Management

Create, read, update, and delete agents. Check that the model an agent names exists and that every tool it lists exists.

An agent is configuration, not a running process. It needs one owner that controls its lifecycle and keeps its fields consistent, whether or not that agent is ever run.

Deleting an agent also deletes its chat sessions and its triggers, because neither can work without the agent. Its runs are kept, because they record things that already happened.

Implementation. `app/modules/agents`. The module is handed the set of valid tool ids and model ids instead of importing the catalogs, so it can tell whether a tool exists without knowing what any tool does. This is not just a convention: a structural test fails the build if this module ever imports tool or model implementation code.

1.3.3 Agent Execution, the orchestrator

Everything involved in running one agent turn: load the agent's configuration, resolve the tools it is allowed to use, send the request to the model, handle whatever tool calls the model makes, and produce both a finished answer and a permanent record of it.

This is the one place that knows how to run an agent from start to finish. That way the two things which can start a run, a chat message and a trigger firing, share every step except the parts that genuinely differ.

It reaches the tools and the LLM through their ports and never learns which implementation it received. It leaves session handling to Session Management, because a scheduled run has no chat session.

Implementation. `app/modules/execution`. Three entry points sit on one shared core: `send_message` and `stream_message` for chat, which differ only in how the answer is delivered, and `run_trigger` for scheduled and manual firing.

1.3.4 LLM Provider

The LLM Provider port defines what every model backend must offer: list the available models, and run one agent turn from the agent's settings plus one user message.

Agent Execution uses the same port whether the mock model or real Gemini is active. Each implementation reports the same four events: a tool call started, a tool call finished, a fragment of text, and the turn finished. Nothing above the port sees a vendor event type.

The port does not include operations such as "summarise" or "make a title". A new chat title is produced by shortening the first message, which needs no model call. The port also receives one message at a time, so earlier turns in the same conversation are not sent back to the model.

Implementation. `app/modules/llm`, with the mock and ADK/Gemini implementations under `providers/`. Which one is active is decided only in `app/container.py`.

1.3.5 Tool Provider

The contract a tool must satisfy, the registry that holds the list of available tools, and the built-in tools themselves.

A tool should be easy to add and easy to reason about on its own: arguments in, result out. Keeping that contract free of any knowledge of agents, runs, or the model is what makes a new tool a self-contained addition.

A tool returns an error result for ordinary problems, such as a bad argument or a failed web request, instead of raising. If something more serious happens, the bridge between the tool and the model catches it, records the call as failed, and returns the error to the model. One failing tool degrades a turn. It never aborts one.

Implementation. `app/modules/tools`, with implementations under `impls/` and `ToolRegistry` as the only module that knows the concrete list.

1.3.6 Knowledge Library

Create, read, update, and delete the documents that the `knowledge_search` tool reads, plus the search that the tool runs against them.

The library is one global collection rather than one per agent. Scoping documents to an agent would require the tool to know which agent called it, and the tool contract carries no caller identity: arguments in, result out. Adding one would push agent awareness into every tool, which is exactly the coupling that contract exists to prevent.

Two paths reach this component, and they are not symmetric. A person managing documents goes through the usual router and service layers. A search goes straight from the tool to the storage port with no service in between, because searching is a read with no policy attached. This is the same reason the tools and models catalogs are read through their ports directly.

Scoring and excerpting are pure functions shared by both storage implementations, so the in-memory store and PostgreSQL rank results identically. The scoring is a linear scan over whole documents: no chunking, no embeddings, no vector index. It suits a library of tens to low hundreds of short documents. Replacing it with full-text or vector search is a change behind the storage port that no caller would see.

Implementation. `app/modules/knowledge`. The tool itself stays in `app/modules/tools/impls/knowledge_search.py` and receives the repository from the composition root.

1.3.7 Run History

The permanent record of every execution: one entry per turn, holding what was asked, what came back, which tools ran, and how the turn ended.

This is the audit trail. It has to stay accurate even after the agent, session, or trigger that produced a run has been edited or deleted. Otherwise "what actually happened" would depend on state that can change later.

Each entry stores the agent's name, model, and system prompt as they were at execution time, not a reference to the agent's current values. Run History is written as a side effect of execution; nothing about when a run happens is decided here.

Implementation. `app/modules/runs`.

1.3.8 Session Management

The identity and title of a chat thread: create one on its first message, give it a title, and let a person rename or delete it later.

A conversation needs an identity that does not depend on any single message in it, so a person can come back to it and so run history can group runs by conversation.

A session does not exist until the first message of a thread is sent. There is no "create empty chat" action, and the title comes from that first message rather than from a prompt to name it. Deleting a session deletes the runs that belong to it, because those runs are the conversation rather than a description of one.

Implementation. `app/modules/sessions`. A triggered run has no session at all, by design (section 4.2).

1.3.9 Trigger Management and the Trigger Scheduler

Create, read, update, and delete triggers; work out when a trigger is next due; and run a background process that watches for due triggers and fires them.

This is the only way to run an agent with nobody present. Separating "what a trigger is" from "the clock that fires it" keeps the trigger routes usable even when firing is switched off.

The scheduler does not run inside a request and response cycle. It is a loop that polls on an interval, independent of any browser request.

Implementation. `app/modules/triggers`, with the scheduler in `scheduler.py` running as a background task for the life of the process.

1.3.10 Persistence

Storage for each entity behind its own port: agents, runs, sessions, triggers, and knowledge documents.

Nothing above this layer should need to know whether storage means a Python dictionary or a PostgreSQL table. Both satisfy the same contract.

Each port has two implementations today: in-memory, which is fast and disposable and used for local development and tests, and PostgreSQL, which is durable. One is chosen per process, never per call.

Implementation. `repositories/memory.py` and `repositories/postgres.py` under each module.

1.3.11 The composition root

The one place that chooses an implementation for each port and hands it to the components that need it.

Keeping these choices in one file makes "which LLM provider and which storage backend are running" a single fact about the process. It is also what forces components to depend on ports rather than on concrete classes.

Services and routers receive their collaborators through dependency injection, either as constructor arguments or as FastAPI dependencies. They never import an implementation module.

Implementation. `app/container.py`. It handles no requests and contains no business rules. Its only job is wiring.

1.4 How a request flows through the system

1.4.1 A chat message

This is the main flow. A person sends a message, the model decides whether a tool is needed, the platform runs the tools it asks for, and the person gets a complete answer plus a permanent record of everything that happened.

Figure 3 shows the request loading the agent and the session, streaming provider events to the browser, recording the finished run, and returning the finished message.

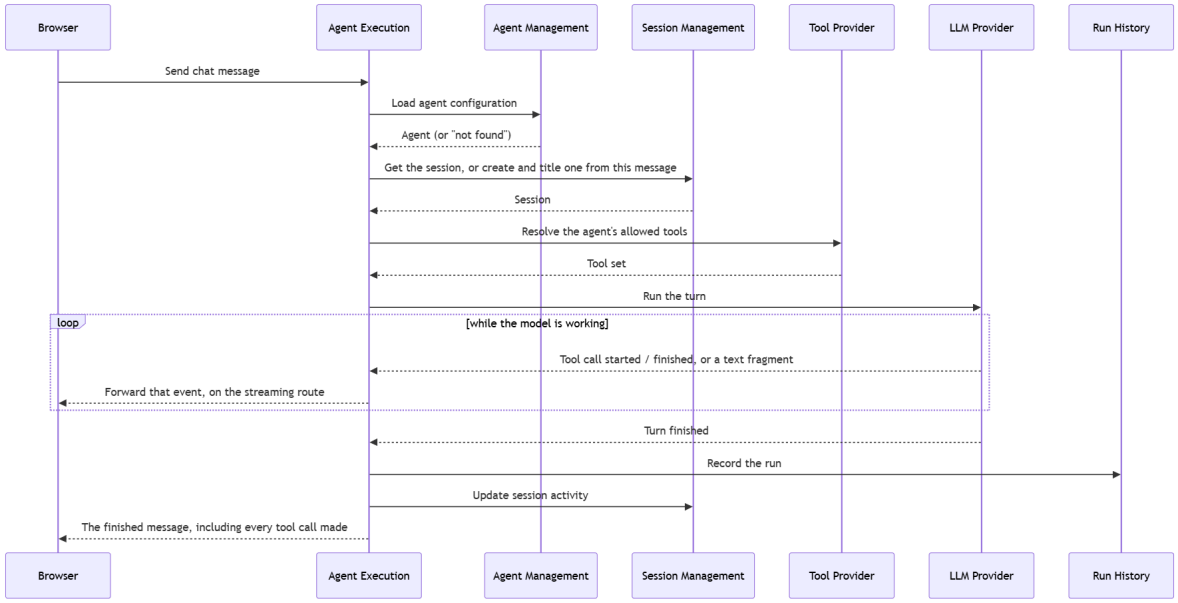

Figure 3: Chat message execution sequence

Three parts of this flow matter.

- **The turn is streamed, but the run record is written only when it is complete.** The browser receives tool calls and model text as they happen, so a long run shows progress. The full run record is stored before the last event is sent. If the stream ends before that event, the client treats the turn as failed rather than showing a partial result as a success.
- **The failure path can be triggered on demand.** A message matching a simple pattern makes one tool call fail, unless the request is marked as a retry. This makes error and retry

behaviour testable without editing code or changing settings.

- **Session handling stays outside the shared core.** Creating, checking, and titling a session apply only to chat. Keeping that work in the chat entry points lets a scheduled trigger use the same execution core without creating a session it does not want.

Implementation. `execution/router.py` calls `ExecutionService.send_message` for the plain JSON route and `stream_message` for the streaming route. Both consume the same provider event stream, typed as `AsyncIterator[RunEvent]`. `event_translator.py` folds those events into the finished message and the run record. The streaming route also writes each event to the browser as newline-delimited JSON while the turn is still running.

1.4.2 A trigger firing

Triggers are the second way to run an agent. Unlike chat, there is no incoming user request: a background process starts the execution.

Figure 4 shows how each scheduler tick finds due triggers, claims each one before running it, calls the shared execution path, and records the outcome.

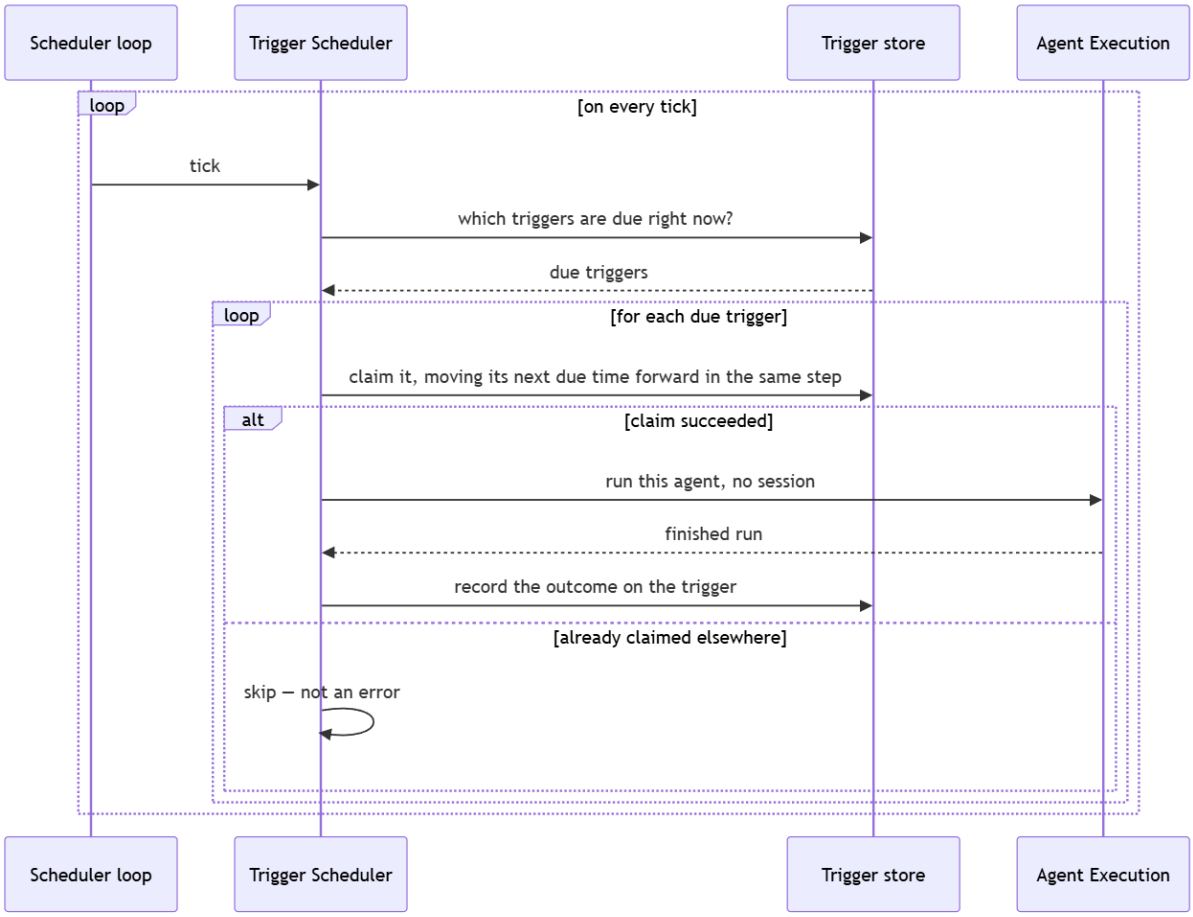

Figure 4: Scheduled trigger execution sequence

Several choices here only make sense once the constraint behind them is visible.

- **After downtime a trigger catches up once, not many times.** The next fire time is computed from the current instant, not from the stale one. No matter how long the scheduler was stopped, each trigger fires at most once when it starts again.

- **A trigger is claimed before it runs.** The claim moves the next fire time forward only if it still holds the value the tick read. Two scheduler instances therefore cannot fire the same trigger at once, and losing the race is not an error, just a skip.
- **One trigger failing does not stop the rest.** A firing that raises is caught and logged, the trigger records an error status, and the tick moves on to the next one.
- **Triggered runs do not appear in chat.** They have no chat session, so they show up only in the trigger's activity log. Conversations stay human-only.

Implementation. `triggers/scheduler.py`, running as a lifespan background task, calling `ExecutionService.run_trigger` – the same orchestrator core chat uses, without the session work.

1.5 Design decisions

The sections above mentioned several choices in passing. This section states the significant ones: what was chosen, what problem it solves, what the realistic alternative was, and what it costs.

Ports between orchestration and external services.

Chosen: Agent Execution depends on interfaces for the LLM, the tools, and storage rather than on specific implementations.

Problem solved: the same execution logic runs against a mock model and an in-memory store during development, and against Gemini and PostgreSQL in a real deployment, with no change to its code.

Alternative: wire the orchestrator directly to one model SDK and one database driver.

Cost: one more layer to read through, and every new provider has to fit the existing interface.

A deterministic mock is the default LLM provider.

Chosen: the platform ships with a scripted mock model, so it runs with no API key and no internet connection.

Problem solved: development, tests, and demonstrations stay repeatable and fast, and cost nothing.

Alternative: require a real model API key from the start.

Cost: the mock proves that the execution flow, the tool calls, and the error paths work. It says nothing about how a real model behaves. Switching to Gemini is a one-variable change, because both sit behind the same port.

Streaming is a second delivery route, not a second execution path.

Chosen: chat has two endpoints that do the same work. One returns the finished result as JSON; the other streams events as they happen. The browser uses the streaming one, so tool calls and model text appear during a long run.

Problem solved: without streaming a user sees nothing for several seconds and assumes the system is broken.

Alternative: offer only request and response, or only streaming.

Cost: two endpoints to maintain, although they share the execution core. The run is stored before the final event is sent, so run history never holds a partial result.

Run history stores a snapshot, not a reference.

Chosen: each run stores the agent's name, model, and system prompt as they were when the run happened.

Problem solved: editing an agent later must not rewrite the history of what it did before.

Alternative: store only the agent id and read its current values when displaying history.

Cost: some data is duplicated, and any new agent field that should appear in history has to be added to the snapshot on purpose.

Deletion rules differ per relationship.

Chosen: deleting an agent removes its sessions and triggers but keeps its runs; deleting a session removes its runs; deleting a trigger keeps its runs; deleting a knowledge document affects nothing else.

Problem solved: these relationships do not mean the same thing. A session and its runs are one object to a user. A run produced by an agent or a trigger is evidence that should outlive them.

Alternative: one database cascade rule for everything.

Cost: the rules live in application code rather than in the schema, so they have to be implemented once per storage backend and tested against both.

The knowledge library is global rather than per agent.

Chosen: one collection of documents that every agent with the `knowledge_search` tool can read.

Problem solved: the tool contract takes arguments and returns a result, with no caller identity. Keeping the library global means the tool needs none.

Alternative: scope documents to an agent and pass the agent id into the tool.

Cost: there is no way to give two agents different document sets, and no document-level access control. Adding either means changing the tool contract for every tool, not just this one.

The trigger scheduler polls instead of using a queue.

Chosen: a background loop checks storage for due triggers on an interval.

Problem solved: scheduled execution works with no extra infrastructure: no message broker, no external scheduler.

Alternative: a message queue or a managed scheduling service.

Cost: a trigger can fire up to one tick late, and scaling the scheduler across instances would need coordination the current design does not have.

Docker Compose is the deployment artifact.

Chosen: the web application, the API, and the database run as three containers managed by Docker Compose.

Problem solved: the whole system starts with one command and no cloud account.

Alternative: build directly against managed cloud services such as Cloud Run and Cloud SQL.

Cost: Compose is right for local use and a small deployment, but it does not exercise what a managed cloud deployment would need.

1.6 Deployment architecture

The platform runs the same way in every environment. The only thing that changes is which provider and which storage backend are configured, through the ports described in section 2.

Local development runs two plain processes on the developer's machine: the backend on port 4000, and the frontend dev server on port 5173, which forwards `/api` requests to it. No container is involved.

The container deployment is the artifact for this deliverable. It brings up three containers together. Figure 5 shows the browser-facing web container, its proxy connection to the internal API, the API's PostgreSQL connection pool, and the optional live Gemini integration.

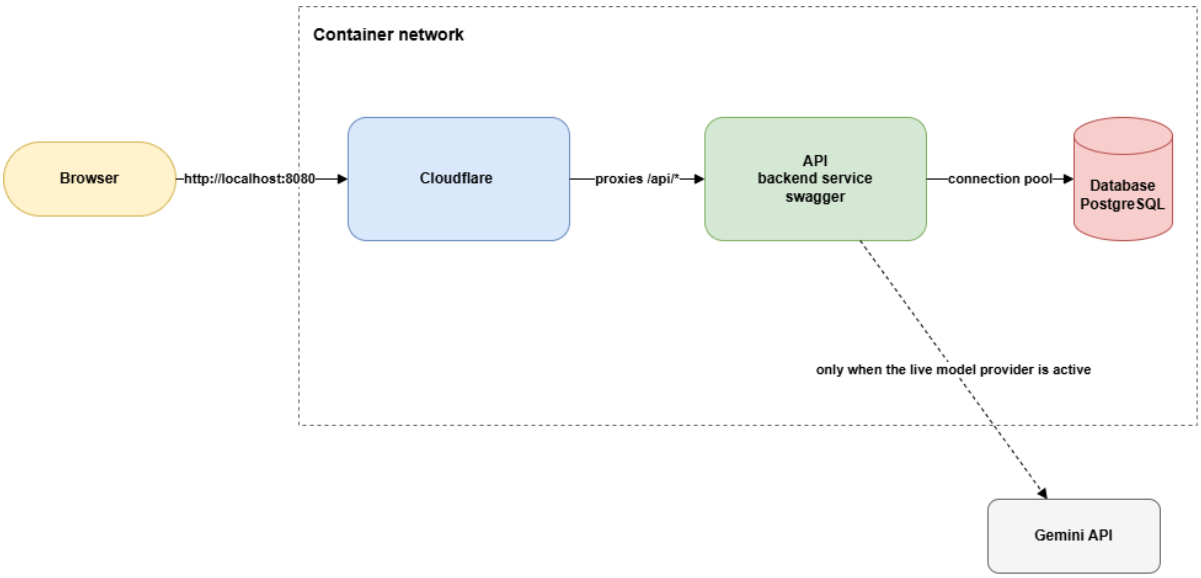

Figure 5: Container deployment architecture

Only the web container publishes a host port. That keeps the browser same-origin with the API: every request goes to the host and port the page came from, so there is no cross-origin configuration to get wrong. The API is not reachable from outside the container network at all.

Startup order is enforced by health checks rather than by a fixed delay. The database must report healthy before the API starts, and the API must report healthy before the web container begins serving. A slow database on a loaded machine therefore delays startup instead of causing an intermittent failure.

Configuration is entirely environment-driven, through a file that is never committed. Only its template is committed, and a fresh clone does not need even that, because every setting has a default in the Compose file. The container stack always runs against real PostgreSQL. What the default configuration changes is the model, which starts as the offline mock. The stack therefore comes up and demonstrates a complete execution flow – tool calls, traces, and history that survives a restart – with no credentials and no cost. Pointing it at the live model provider is one variable plus an API key, with no code change. That is the payoff of the ports decision in section 5.

Access control is a shared password in front of nginx, or nothing. The API has no authentication of its own. The web container reads a `WEB_PASSWORD` setting; when it is empty, which is the default, the site is open, and `docker compose up --build` needs no setup at all. When it is set, nginx puts a login page in front of both the UI and `/api`, and checks a cookie on every request. The check runs in nginx, so a command-line client is gated too, not just the browser. This is one shared password with no per-person identity and no logout: changing it and recreating the web container revokes access for everyone at once. It is enough to keep a demonstration instance from being an open LLM proxy. It is not a user model, and section 7 treats that gap as open.

Implementation. `AgentPlatform-BackEnd/deployment`, driven by `docker compose up --build`. Configuration lives in `deployment/.env`, copied from the committed `.env.example` and needed only to change a default. The full variable table is in chapter 2, section 5.2.

1.6.1 Mapping to a cloud-native target

Each container is built independently and makes no assumption about its host, so moving this deployment onto managed cloud infrastructure is a substitution rather than a redesign.

Container	Cloud equivalent
Web (reverse proxy and static client)	A managed container service, or static hosting plus a CDN for the client bundle
API (backend service)	A managed container service. It is stateless and scales on request volume, with one caveat: the trigger scheduler runs as a background loop inside the API process (section 4.2) and assumes a single active instance, so scaling out would need the scheduler moved to a singleton job runner or a managed scheduler product
Database	A managed PostgreSQL service
Environment configuration file	A managed secrets service, injected as environment variables at deploy time
Password gate in nginx	An identity-aware proxy or the platform's own access layer

This mapping is not implemented here. Docker Compose is the delivered artifact. Nothing in the current design blocks the move.

1.7 Tradeoffs, limitations, and risks

This section collects the costs of the choices above in one place, rather than leaving each of them implicit in its own section.

- **The trigger scheduler assumes a single active instance.** The claim step (section 4.2) stops two instances from firing the same trigger twice, but nothing moves scheduling off the API process. Scaling the API horizontally would either leave every instance's loop racing harmlessly and wastefully, or require the scheduler to be pulled out into its own singleton process. This is the clearest structural risk in the design if the platform outgrows one instance.
- **There is no user model.** The password gate in section 6 is one shared secret in front of the whole site. There is no login identity, no per-user data isolation, and no permission model. Everyone who knows the password sees and can delete everything. This is appropriate for a proof of concept and wrong for anything holding more than one person's data. It is the first gap to close before a real deployment.
- **A chat thread is a record of a conversation, not a memory of one.** Each turn sends the model the agent's system prompt and the current message only. Earlier turns in the same session are not replayed. A session groups runs so a person can return to a thread and read it back, and it is durable, but the agent does not remember what was said three messages ago, so a follow-up such as "and the other one?" will not resolve. Adding memory is a change to what the execution core sends the provider, not to the shape of the system: the thread is already stored, already ordered, and already retrievable. It is the most visible functional gap between this platform and a product-grade assistant.
- **Knowledge search is a linear scan, and the library is global.** There is no chunking, no embedding, and no relevance model beyond term overlap. That is fine for tens to low hundreds of short documents and will not hold for a large corpus. Because the library is global, there is also no way to give two agents different documents.

- **The default configuration is shaped for demonstration, not production.** The mock model is what lets the platform boot with no external dependency, which is the right choice for this deliverable. It also means the failure modes, the latency, and the cost of a real model stay invisible until someone switches the provider on purpose. Anyone relying on this platform in earnest needs to exercise the live-provider path before trusting its behaviour.
- **Streaming adds a failure mode with no status code.** Once the response has started, the status is already 200, so a provider failure part-way through reaches the browser as an error event inside the stream rather than as a 502. Every client therefore has to handle two kinds of failure, and one of them is only visible to a client that reads the body to the end. A dropped connection is invisible to the server as anything except a stream nobody finished reading.
- **Snapshotting ties the shape of run history to the shape of an agent.** Any new agent field that should also be preserved in history has to be added to the snapshot by hand. Nothing keeps the two in sync automatically, so this is a place a future change could quietly go stale.
- **A single database instance is a single point of failure.** The current deployment has no replication, no failover, and no backup plan beyond what a managed service would provide if one were adopted. Durability here means "data survives a container restart" and nothing more.

Chapter 2

Technical Documentation

Chapter 1 explains in plain terms what the platform is and why it is shaped the way it is. This chapter is the engineering reference behind that explanation: the component boundaries as they are actually implemented, the HTTP contract field by field, the persistent data model, the two extension interfaces, and the commands used to deploy and develop against it.

Everything here was checked against the code in this repository and against a running instance, not against the design notes that came before it. Where an older plan or specification disagrees with the running system, the running system is what is documented.

2.1 Overall architecture

2.1.1 System at a glance

The platform lets a person define agents and run them two ways: interactively through chat, or unattended on a schedule. An agent is a name, an icon, a description, a model, a system prompt, and a list of allowed tools. Every completed turn, chat or scheduled, writes one run record holding the question, the answer, the ordered tool calls with their arguments, results and durations, the latency, the status, and a snapshot of the agent configuration that produced it.

The backend keeps agent configuration, execution orchestration, model integration, tool integration, knowledge storage, and persistence apart, because those concerns change for different reasons and at different times. A new model provider should not touch agent management. A new tool should not need a new route. Changing the storage backend should not alter a business rule. Those same boundaries are what make the default configuration – a deterministic mock model over an in-memory store – a complete, runnable system that needs no credentials and no database.

Figure 6 shows the components and the direction of the dependencies between them.

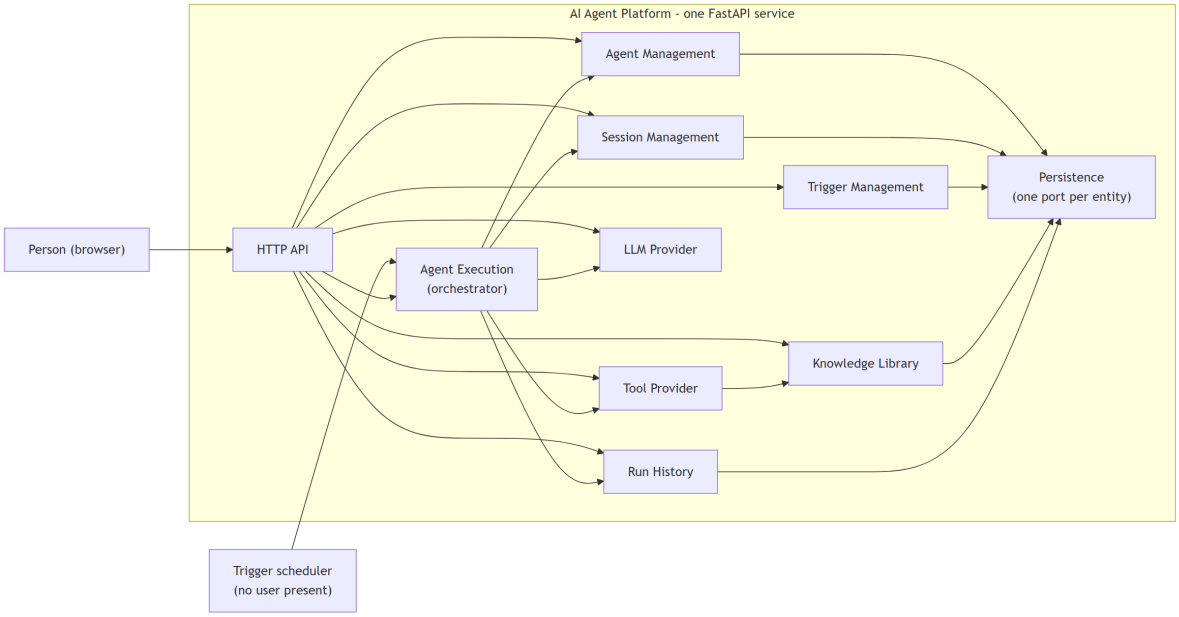

Figure 6: Overall system architecture

Design decisions.

- **One deployable service.** The whole backend is a single FastAPI application. This keeps the proof of concept simple to operate, at the cost of not being able to scale or deploy one component on its own.

- **Business components depend on interfaces, not implementations.** Every collaborator that could plausibly be swapped – storage, the model, the tool catalog – is reached through a Python `Protocol`. That is what allows in-memory and PostgreSQL stores, a deterministic and a live model provider, and isolated tests, with no change to the code that uses them.
- **Chat and triggers share one execution core.** Scheduled execution is not a second engine with its own tool handling, failure handling, or audit behaviour. The two entry points differ only in what surrounds the shared core.
- **Provider events are normalised at the boundary.** Nothing above `LLMProvider` sees a vendor event type. The price is that each adapter must translate its SDK's native event model into the platform's four-event vocabulary, preserving ordering, timing, and result shape.
- **The composition root is one file.** `app/container.py` is the only module that picks a concrete repository or provider, and `app/config.py` is the only module that reads the environment. If either choice could be made in a second place, "which provider is running" would stop being a single fact about the process.

Implementation mapping.

Conceptual component	Where it lives
Web client	React 19 and TypeScript on Vite, in `AgentPlatform-FrontEnd/client`
HTTP API	FastAPI application built by `create_app()` in `AgentPlatform-BackEnd/server/app/main.py`
Agent Management	`app/modules/agents`, business rules in `AgentService`
Agent Execution	`ExecutionService` and `event_translator.py` in `app/modules/execution`
LLM Provider	`LLMProvider` in `app/modules/llm/provider.py`; implementations `MockLLMProvider` and `AdkGeminiProvider`
Tool Provider	`Tool`, `ToolSchema`, `ToolRegistry`, and the four tools under `app/modules/tools`
Knowledge Library	`app/modules/knowledge`: document CRUD, plus the scoring and excerpting both stores share
Run History	`app/modules/runs`
Session Management	`app/modules/sessions`
Trigger Management and scheduler	`app/modules/triggers`, split between `TriggerService` and `TriggerScheduler`
Persistence	A repository `Protocol` per module, with memory and PostgreSQL implementations under each module's `repositories/`; SQL in `app/core/schema.sql`
Composition root	`app/container.py`

The backend is organised as vertical slices under `app/modules/`, not as global `controllers/`, `services/`, and `models/` folders. Everything genuinely cross-cutting lives in `app/core/`: the error envelope, the body-size limiter, the Express-compatible JSON parser, the connection pool, identifier generation, the injectable clock, and camelCase wire conversion.

`tests/test_structure.py` checks the boundaries mechanically. It imports every module in the tree, which is why the optional Google ADK import is deferred into the functions that use it, and it asserts that nothing under `app/modules/agents/` imports the tool or model catalogs.

2.1.2 Agent execution

`ExecutionService` coordinates one agent turn. It loads the agent's current configuration, turns the tool ids stored on it into callable tools, builds a provider-neutral run specification, consumes

the provider's event stream, folds those events into a user-facing message and a run record, and appends the run to history.

This is the one component that depends on nearly everything else, and that is its job. Spreading the same coordination across the router, the trigger scheduler, and each provider adapter would give three places the chance to disagree about what a turn is.

Figure 7 traces the router's request through agent lookup, tool resolution, provider events, run persistence, and the final response.

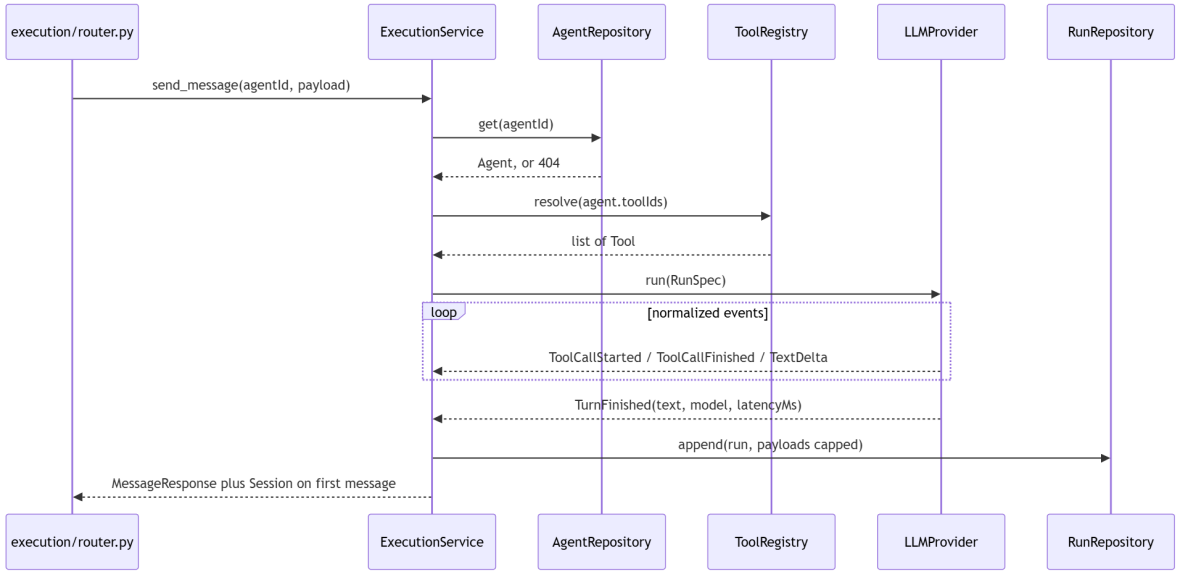

Figure 7: Agent execution flow

How a turn is assembled.

1. A chat request or a trigger firing names an agent and supplies a message.
2. The agent is loaded. An unknown id is a 404 before anything else happens.
3. `ToolRegistry.resolve()` turns the stored `toolIds` into tools, in the stored order.
4. A `RunSpec` carries agent identity, display name, model, system prompt, resolved tools, the message, the retry flag, and the optional session id, trigger id, and timezone.
5. The provider runs one complete agent turn, including any tool loop the model drives, and emits `ToolCallStarted`, `ToolCallFinished`, `TextDelta`, and finally `TurnFinished`.
6. `EventAccumulator` pairs each start with its finish by `call_id`, keeps invocation order, and derives status. A call is `error` if it reported one, and the turn is `error` if any call failed. A start with no matching finish is recorded as a failed call carrying the message `The provider ended the turn before this call reported a result.`
7. The answer is `TurnFinished.text`, falling back to the joined text deltas when the provider sends no final text.
8. Tool arguments and results larger than `LOG_PAYLOAD_MAX_BYTES` are replaced **in the stored row only** with `{"truncated": true, "bytes": N, "preview": "..."}`. The HTTP response is never altered, so the trace on screen always shows what the tool really returned.
9. The run is appended. Nothing ever updates an existing run.

Design decisions.

- **The provider port represents one agent turn, not one model completion.** The tool-calling loop belongs to the model SDK. Modelling a single completion would have forced that loop up into `ExecutionService` and made every adapter reimplement it differently.

- **A run snapshots `agentName`, `model`, and `systemPrompt`.** Agents are mutable. Resolving those through the current agent at read time would let an edit rewrite the historical explanation of a past run.
- **Tool failures are data; provider failures abort the turn.** A failed tool call is stored and returned inside a completed turn, because the model saw it and answered around it. A provider failure raises `ProviderError`, answers 502, and appends **no** run, because there is no complete event sequence to record.
- **Upstream error text never reaches the response.** `llm/failures.py` classifies the raw exception into one of five short sentences – quota, credentials, timeout, unavailable, unknown. The vendor's own message, which carries billing URLs and sometimes the prompt, goes to the log instead.
- **Chat-specific work stays outside the shared core.** Session creation, ownership checks, titling, and activity updates live in the chat entry points. `run_trigger()` creates no session, so a firing can never appear in the chat sidebar.

Implementation mapping. `ExecutionService._execute()` is the shared core. `send_message()` and `stream_message()` add the chat concerns around it. `run_trigger()` supplies a trigger id and a timezone and omits the session. `event_translator.py` holds `EventAccumulator` and `translate()` and knows nothing about any specific provider.

2.1.3 Conversations and scheduled execution

A session is the identity and title of a human conversation. A trigger is a stored message plus a schedule. Both produce runs, but only chat produces sessions.

The first message of a chat creates its session and titles it by deterministic truncation of that message. There is no model call, so the first turn costs nothing extra and the title is stable and works offline. Later messages must send the returned `session.id`. The session must exist, or the request is a 404, and it must belong to the agent named in the URL, or the request is a 400. Both checks run before the provider call and before the run is appended, so a rejected send leaves nothing behind.

The scheduler is one `asyncio` task started by the FastAPI lifespan. Figure 8 shows how each tick finds due triggers, claims them, runs the agent, and records the outcome.

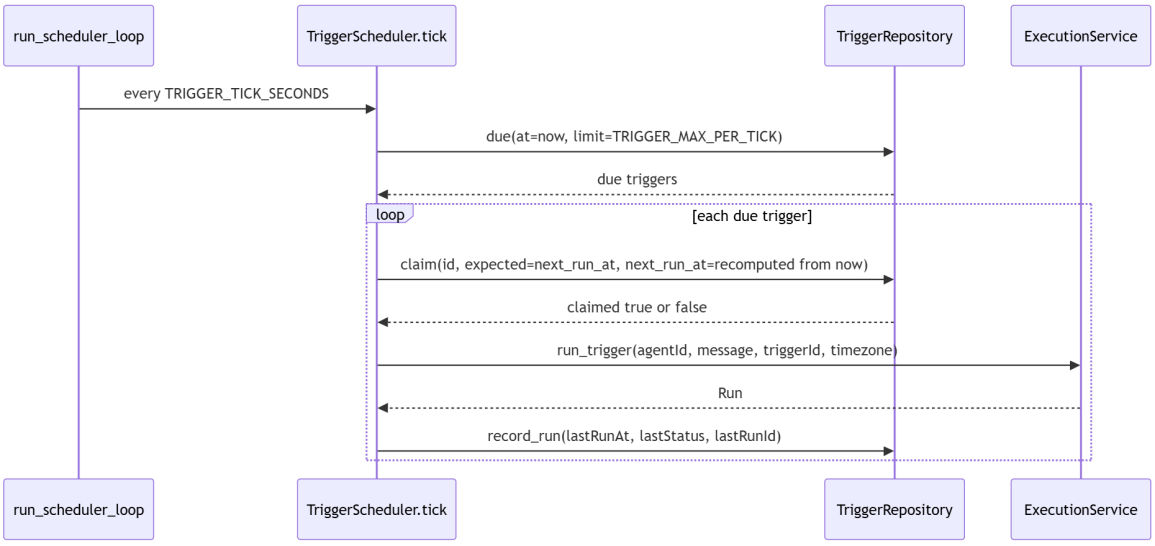

Figure 8: Trigger scheduler flow

Design decisions.

- **Missed firings collapse instead of replaying.** The next fire time is computed from the current instant, not from the stale `next_run_at`. After an outage a trigger fires once and resumes from now, rather than emitting a backlog of catch-up runs.
- **Claiming is a compare-and-swap.** `claim()` moves `next_run_at` forward only if it still holds the value the tick read. A process that loses the race simply does not fire; a lost claim is not an error.
- **Firings are awaited one at a time.** With a single scheduler task this is also what stops a trigger from overlapping its own previous firing when a run outlasts its interval.
- **The clock is separable from the engine.** `tick()` is the whole engine and the loop around it is a wrapper, so moving scheduling into a dedicated worker later is a change of entry point rather than a redesign.
- `TRIGGERS_ENABLED=false` **stops the clock only.** Every trigger route keeps working, including fire-now, which is independent of both the flag and the scheduler.

Implementation mapping. Session lifecycle is split between `ExecutionService._prepare_message()` (create, validate, title) and `SessionService` (list, rename, delete). Trigger CRUD and fire-now are `TriggerService`. Polling and scheduled firing are `TriggerScheduler.tick()` and `run_scheduler_loop()`. Next-fire computation is a pure function in `triggers/schedule.py` with no clock and no I/O, which is what lets the only subtle logic in the feature be read on its own.

2.1.4 Client and API boundary

The browser application is a separate single-page app that reaches the backend only through JSON over `/api`.

`src/lib/api-client.ts` is the only frontend module that calls `fetch`. It owns URL construction, JSON encoding, the NDJSON stream reader, and the translation of the error envelope into a typed `ApiError` carrying status, code, and message. A network failure becomes `ApiError(0, 'network_error', ...)`, so callers handle a dead backend and a rejected request the same way. `src/lib/api-host.ts` resolves the base URL: blank in development so Vite proxies `/api`, or `VITE_API_HOST` when the API is hosted elsewhere.

State lives in hand-written hooks rather than in a query library. `useAgents` and `useSessions` are each mounted once through a provider so every surface shares one list, and both apply writes optimistically with rollback held per entity rather than per list snapshot. `useChat` holds threads as `Record<sessionId, Message[]>`, consumes the streaming chat route event by event, and rebuilds a reopened conversation from `GET /api/runs?sessionId=` through `lib/run-to-messages.ts`. `useKnowledge` is deliberately not a provider: only the Knowledge page reads it, so it is a plain hook.

Design decision. The API is authoritative for everything persisted; the browser holds presentation and interaction state only. A chat thread is reconstructed from sessions and runs rather than being durable browser state, which is why history survives a reload on a different machine and why a reloaded thread shows exactly what was recorded.

2.2 API reference

The base path is `/api`. In development the FastAPI application serves it directly on port 4000 and Vite proxies to it from 5173. In the container stack nginx publishes port 8080 and proxies `/api/` to `api:4000` on the internal network, so the browser is always same-origin and never issues a CORS preflight.

All bodies are JSON. **Wire JSON is camelCase and Python identifiers are**

snake_case: every schema crossing the boundary inherits `WireModel`, which applies a camelCase alias generator. A field that misses that base class reaches the frontend as `undefined` with no error at all, so the symptom is an empty section of UI rather than a failure.

Three cross-cutting behaviours apply before any route runs:

- **Body size.** `BodySizeLimitMiddleware` rejects a request whose `Content-Length` exceeds `MAX_BODY_BYTES` (default 262,144) and also counts the streamed body, so a chunked request cannot bypass the header check. A non-numeric `Content-Length` is a 400 with `Malformed Content-Length header.` The middleware sits inside CORS so a 413 still carries `Access-Control-Allow-Origin`.
- **JSON parsing.** `core/http.json_body` reproduces `express.json()` on purpose, because the contract this service replaced was an Express application. A body that is absent or not sent as `application/json` parses as `{}` rather than failing. Strict mode requires the first non-whitespace byte to open an object or array. `NaN` and `Infinity` are rejected, and a lone UTF-16 surrogate is rejected at the door rather than raising during response serialisation after the write already happened. All of those answer `Malformed JSON body.`
- **String lengths are measured in UTF-16 code units**, matching JavaScript's `String .length` rather than Python's `len()`, so an emoji costs 2 and the frontend's live character counters agree with the server.

2.2.1 Error envelope

Every failure, without exception, serialises as:

```
{ "error": { "code": "bad_request", "message": "..." } }
```

FastAPI's defaults work against this in two places, and both have an explicit handler: request validation failures default to 422 with FastAPI's own body shape, and unmatched routes raise `StarletteHTTPException`, whose default body is `{"detail": ...}`.

Status	code	When
400	`bad_request`	Validation failure, malformed JSON, or a request-validation failure FastAPI would return as 422
404	`not_found`	Missing resource, or an unmatched method and path. An unsupported method answers 404 rather than 405, with `No routeforPUT/api/agents`
413	`payload_too_large`	Body exceeds `MAX_BODY_BYTES`; the message is always `Req uestbodyistoolarge.`
502	`provider_error`	The model provider failed or timed out; the message is one of five classified sentences
500	`internal_error`	Unhandled exception; the public message is always `Someth ingwentwrongontheserver.`

The frontend parses this shape in `api-client.ts` and shows `message` directly, which is why messages are written as sentences a person can act on.

2.2.2 Health

Method	Path	Response
GET	`/api/health`	`{"status":"ok","mode":"mock"|"live"}`

`mode` reports the configured `LLM_PROVIDER` and nothing more. It does not prove that a credential is valid or that a provider call has ever succeeded. With `LLM_PROVIDER=adk_gemini` and an empty key the service still starts cleanly and reports `live`, and the failure appears as a 502 on the first chat request. The sidebar status pill polls this every five minutes.

2.2.3 Agents

An agent is the mutable configuration used by future executions.

Method	Path	Status	Response
GET	`/api/agents`	200	`Agent[]`, newest `updatedAt` first
POST	`/api/agents`	201 / 400	`Agent`
GET	`/api/agents/{agentId}`	200 / 404	`Agent`
PATCH	`/api/agents/{agentId}`	200 / 400 / 404	`Agent`
DELETE	`/api/agents/{agentId}`	204 / 404	—

`Agent`

Field	Type and constraints	Ownership
`id`	string, prefixed `agent_`	Server-owned
`name`	string, at most 120 characters	Writable
`icon`	string, at most 32 characters	Writable
`description`	string, at most 2,000 characters	Writable
`model`	string, at most 64 characters, must be a known model id	Writable
`systemPrompt`	string, at most 20,000 characters	Writable
`toolIds`	array of known tool ids, no duplicates, no longer than the registered catalog	Writable
`status`	`"active"` or `"draft"`	Writable
`createdAt`, `updatedAt`	ISO-8601 timestamps	Server-owned

Write validation runs in a fixed order, and the first failing rule decides the response:

1. The six string fields, in the order `name`, `icon`, `description`, `model`, `systemPrompt`, `status` – type first, then length.
2. `model` against the known model set.
3. `status` against `{"active", "draft"}`.
4. `toolIds` – array, then all strings, then catalog size, then duplicates, then unknown ids.

Representative messages, asserted character for character by the contract tests: `name must be at most 120 characters.`, `Unknown model "x".`, `Unknown status "x".`, `toolIds must not contain duplicates.`, `Unknown tool ids: a, b.`

An absent body is treated as `{}`. A non-object body answers `Request body must be a JSON object.` Keys outside the seven writable fields, including `id`, `createdAt`, and `updatedAt`, are **dropped silently** from a create or patch rather than rejected, so a client that echoes a whole agent back does not have to strip the server-owned fields first.

`POST /api/agents` with an empty body is valid and produces a usable draft: name `New agent`, the default model, empty description and prompt, no tools, status `draft`. Both timestamps come from one clock read, so a new agent's `createdAt` and `updatedAt` are identical rather than microseconds apart. `updatedAt` moves on every patch, including an empty one.

`PATCH` looks the agent up **before** validating the body, so a patch to a deleted agent answers 404 rather than a validation error. Deleting an agent also deletes its sessions and its triggers, and keeps its runs. Section 3.2 sets out why.

2.2.4 Tools and models

Two read-only catalogs describing what may be attached to an agent. Both routes read their injected port directly and have no service layer, because there is no business rule between the request and the catalog.

Method	Path	Response
GET	`/api/tools`	`ToolSchema[]` — `{id,label,description,params}`, in registration order
GET	`/api/models`	`ModelInfo[]` — `{id,label}`

Registration order is part of the contract rather than an accident: it is the order the frontend tool picker renders. The model catalog currently holds one entry, `gemini-3.1-flash-lite` (`Gemini 3.1 Flash Lite`), which is also the default model applied to a new agent.

2.2.5 Chat and execution

Two routes execute one complete agent turn. They do identical work and differ only in how the result reaches the client.

Method	Path	Status	Response
POST	`/api/chat/{agentId}/messages`	200 / 400 / 404 / 413 / 502	`{message,session?}` as JSON
POST	`/api/chat/{agentId}/messages/stream`	200 / 400 / 404 / 413	`application/x-ndjson` event stream

The frontend uses the streaming route so tool activity and model text appear as they happen. The JSON route is the simpler contract, and most of the chat contract tests use it. One test covers the stream end to end, asserting that the run is persisted and that its answer matches the `done` event.

2.2.5.1 Request body

```
{ content: string, retry?: boolean, sessionId?: string }
```

- `content` is required, trimmed, must be non-empty after trimming, and at most 10,000 UTF-16 code units. The trimmed value is what is measured, validated, and stored.
- Omitting `sessionId` starts a new chat, and the response carries the created `session`.
- Later messages send the returned `session.id`. An unknown session is a 404. A session belonging to another agent is a 400 reading `Session "sess_x" does not belong to agent "agent_y".`
- On every message after the first, `session` is **absent** from the response rather than null. The route sets `response_model_exclude_none`, so clients test `if (response.session)`.
- `retry: true` suppresses the deterministic failure path for that request only. It is request metadata, never server state, so identical messages from different clients stay independent.

Validation order is content type, then blankness, then length, then `retry`, then `sessionId`, so a body breaking two rules reports the earlier one. `retry` is type-checked rather than coerced: `retry: "yes"` is a 400, not `true`.

2.2.5.2 Response shapes `MessageResponse` carries `id`, `role: "assistant"`, `content`, `toolCalls[]`, `model`, `latencyMs`, `status: "done" | "error"`, and `createdAt`. The turn's status is `error` when any tool call failed, even though the answer itself is present.

Each `ToolCall` carries `id`, `toolId`, `args`, `durationMs`, `status: "ok" | "error"`, plus `result` on success and `error` on failure. The unused key is **absent**, not null, because the trace rail renders "no result" differently from a result that is literally `null`.

2.2.5.3 Streaming events Newline-delimited JSON, one object per line, sent with `Cache-Control: no-cache` and `X-Accel-Buffering: no`. nginx sets `proxy_buffering off` on `/api/` so the stack does not hold events back.

`type`	Payload	Meaning
`session`	`session`	Sent first, and only on the request that created a session
`textDelta`	`text`	A fragment of model text
`toolStarted`	`call` with `status:"running"` and `durationMs:0`	A tool invocation began
`toolFinished`	`callId`, `result`, `error`, `durationMs`, `status`	That invocation completed or failed
`done`	`message`	The same `MessageResponse` the JSON route returns
`error`	`error` with `code` and `message`	The turn failed after headers were already sent

Validation and agent lookup happen **before** the response starts, so a bad request still gets a normal status code and the error envelope. Once the stream has opened the status is already 200, which is why a later failure arrives as an `error` event inside the body; the client turns it back into an `ApiError`. A stream that ends without a `done` event is treated as a failure by the client rather than as a successful empty turn.

2.2.5.4 The deterministic failure path `MockLLMProvider` exists so the full flow – tool calls, trace, failure, retry, persisted run – is reachable with no API key, and so the contract tests are repeatable. Its behaviour is part of the contract:

- It calls at most **two** tools, taken in the agent's configured order, and skips any tool with no fixture.
- Content matching `/\bfail/i` is a word-boundary prefix match: `fail`, `failure`, and `failing` trigger it; `unfail` does not. When it triggers and the agent has at least one fixture-backed tool, the **last attempted** call fails with `connection refused after 800ms` and a `durationMs` of 812, and the answer explains the failure.
- `retry: true` suppresses that, so the same message succeeds on the next attempt.
- `latencyMs` is the sum of the tool durations plus a fixed 180 ms of model time.
- It emits no `textDelta` events, so over the streaming route a mock turn shows tool events followed by `done`. Live Gemini streams text, so the two look different on screen by design.
- `knowledge_search` **is the one tool the mock does not fake.** It really executes against the document library, so a document a user uploaded is visible in chat without a Gemini key.

Only the value is real: the duration stays the fixture's, so run totals remain deterministic. The fixture query is fixed, so against the seeded library the demo output does not move.

A retry appends a **new** run and leaves the failed one in place. The live thread replaces the failed turn with its retry, so a reloaded thread shows both attempts where the live one showed only the success. That is intended: the run log is an audit trail, and a failure is never rewritten out of it.

2.2.6 Sessions

A session is the identity and title of one chat thread. It carries no agent name, icon, or model. Those are snapshotted on runs, because they are an audit concern, whereas a session is navigation and has to reflect a rename immediately.

Method	Path	Status	Response
GET	`/api/sessions?limit=`	200	`Session[]`, newest activity first; `limit` 1–200, default 50
PATCH	`/api/sessions/{sessionId}`	200 / 400 / 404	`Session`
DELETE	`/api/sessions/{sessionId}`	204 / 404	—

`Session`: `id` (prefixed `sess_`), `agentId`, `title`, `createdAt`, `updatedAt`.

A session is created only by the first message of a chat and is titled from that message: whitespace collapsed, cut at a word boundary near 60 characters with an ellipsis, hard-cut for a single long word, and the placeholder `New chat` for a message with no visible content. Its `updatedAt` is touched by every message, which is what orders the sidebar.

Rename accepts `{ title: string }`. The title is type-checked, capped at 120 UTF-16 code units, then trimmed. A missing or blank title answers `title is required`. Other keys are dropped silently.

Validation runs **before** the lookup here, which is the opposite of `PATCH /api/agents/{id}`. The two differ only when the id is unknown *and* the body is invalid: sessions answer 400 and agents answer 404. Sessions are renamed from a list the client already holds, so a bad body is the likelier mistake and naming it is the more useful answer. This asymmetry is recorded rather than aligned.

Deleting a session deletes its runs. That cascade lives in `SessionService`, not in a foreign key, so both storage backends behave identically.

2.2.7 Triggers

A trigger stores an agent, a schedule, and one literal message sent verbatim on every firing.

Method	Path	Status	Response
GET	`/api/triggers?agentId=&limit=`	200	`Trigger[]`, newest `updatedAt` first; `limit` 1–200, default 50
POST	`/api/triggers`	201 / 400 / 404	`Trigger`; 404 when `agentId` is unknown
GET	`/api/triggers/{triggerId}`	200 / 404	`Trigger`
PATCH	`/api/triggers/{triggerId}`	200 / 400 / 404	`Trigger`

Method	Path	Status	Response
DELETE	`/api/triggers/{triggerId}`	204 / 404	—; keeps historical runs
POST	`/api/triggers/{triggerId}/run`	200 / 404	`Run` — fire once, now

`Trigger`

Field	Type and constraints
`id`	Server-owned, prefixed `trg_`
`agentId`	Non-empty string; required on create, and the agent must exist
`name`	String, at most 120 characters; defaults to `Newtrigger`
`message`	String, at most 20,000 characters; must be non-blank on create
`kind`	`"interval"` or `"daily"`; required on create
`intervalMinutes`	Integer of at least 1 for `kind="interval"`; null otherwise. `true` is rejected, not coerced
`timeOfDay`	`HH:MM` in 24-hour form for `kind="daily"`; null otherwise
`weekdays`	Integers 0–6 with Monday as 0, no duplicates; empty means every day, and an interval trigger always stores empty
`timezone`	Valid IANA zone name, at most 64 characters; defaults to `UTC`
`enabled`	Boolean; defaults to true
`nextRunAt`	Server-owned; null while disabled
`lastRunAt`, `lastStatus`, `lastRunId`	Server-owned outcome of the most recent firing
`createdAt`, `updatedAt`	Server-owned

A patch is validated **against the current trigger**, not against the body alone, because "an interval trigger needs `intervalMinutes`" is a question about the trigger's effective kind. Switching `kind` to `interval` without sending `intervalMinutes` therefore succeeds only when the stored trigger already has a usable interval. Schedule normalisation then blanks the fields the other kind owns, so an interval trigger cannot keep a stale `timeOfDay` and read back a schedule that is not the one that fires.

`nextRunAt` is recomputed only when the body actually contains a scheduling key: `kind`, `intervalMinutes`, `timeOfDay`, `weekdays`, `timezone`, or `enabled`. Testing the normalised result instead would recompute on every patch, which for an interval trigger would push the next firing out by a full interval every time somebody renamed it.

Daily schedules handle both daylight-saving edges explicitly. A wall-clock time that occurs twice picks the first pass. A time that does not exist at all is walked forward to the first instant that does.

Server-owned fields are dropped silently from create and patch bodies, the same way agents drop `id` and `createdAt`. `PATCH` looks the trigger up before validating, so a patch to a deleted trigger answers 404.

Fire-now works whether or not the trigger is enabled and whether or not the scheduler is running, because firing once before turning a schedule loose is the main reason it exists, and it does **not** reschedule. A triggered run, however it was started, has `sessionId: null` and its `triggerId` set.

2.2.8 Runs

A run is the historical record of one completed execution. Runs are appended, never updated. The only ways one disappears are deleting the agent's run history, deleting its session, or deleting the run itself.

Method	Path	Status	Response
GET	/api/runs?agentId=&limit=	200	Run[] for one agent
GET	/api/runs?sessionId=&limit=	200	Run[] for one chat
GET	/api/runs?triggerId=&limit=	200	Run[] for one trigger
GET	/api/runs/export?agentId=	200	Every run for one agent, streamed as a JSON array file
GET	/api/runs/{runId}	200 / 404	Run
DELETE	/api/runs?agentId=	204 / 400	Deletes every run for one agent

limit is 1–200, default 50, and results are newest first. At most one of agentId, sessionId, and triggerId may be supplied; omitting all three lists runs across every agent. The agentId plus sessionId case answers Pass agentId or sessionId, not both. verbatim, because that message predates triggers. Any combination involving triggerId answers Pass at most one of agentId, sessionId, triggerId.

DELETE is asymmetric with GET on purpose. A missing agentId on the read means "every agent", but on a delete that default would erase the entire history, so it answers agentId is required.

/api/runs/export is the download behind the run-log action in the agents table. It takes agentId only, applies no limit, and streams the rows as they are read rather than building the whole array in memory, so exporting a long history does not scale with process memory. It sets Content-Disposition: attachment with the filename run-logs.json; the browser replaces that name with one built from the agent before saving.

Run: id, agentId, agentName, model, systemPrompt, userMessage, answer, status, error, latencyMs, sessionId, triggerId, createdAt, and toolCalls[].

Each RunToolCall carries id, seq, toolId, args, result, error, durationMs, and status. seq is the invocation order, assigned when the call started rather than when it finished, so a provider that runs tool calls concurrently still persists them in the order the model asked for them.

agentName, model, and systemPrompt are execution-time snapshots, which is what makes this endpoint an audit trail rather than a join against current configuration.

2.2.9 Knowledge documents

The library that knowledge_search reads. It is one global collection: every agent with the tool attached searches all of it. There is no per-agent scoping, because the tool interface carries no caller identity – execute(**kwargs) receives the model's arguments and nothing else.

Method	Path	Status	Response
GET	/api/knowledge/documents?limit=	200 / 400	KnowledgeDocumentSummary[], newest updated first; limit 1–500, default 100
POST	/api/knowledge/documents	201 / 400	KnowledgeDocument
GET	/api/knowledge/documents/{documentId}	200 / 404	KnowledgeDocument
PATCH	/api/knowledge/documents/{documentId}	200 / 400 / 404	KnowledgeDocument
DELETE	/api/knowledge/documents/{documentId}	204 / 404	—

KnowledgeDocument: id (prefixed doc_), title, body, source, createdAt, updatedAt.

KnowledgeDocumentSummary: the same fields with body replaced by preview (the first 200 characters, whitespace collapsed) and sizeBytes. The list route returns summaries and only the detail

route returns the text, because a library of fifty 100 KB documents would otherwise ship several megabytes on every page load.

`source` is one of `typed`, `upload`, or `seed`. It is accepted on create, defaulting to `typed`, and **dropped from a patch**, so a document cannot be relabelled `seed` after the fact and make the sample badge in the interface lie.

Uploads arrive as ordinary JSON. The browser reads a `.txt` or `.md` file with `FileReader` and posts its text, so there is no multipart route and no `python-multipart` dependency.

Writes accept `title` and `body`. `id`, `createdAt`, and `updatedAt` are dropped silently, as they are on agents. Validation order is title type, title length, body type, body length, source, then the required checks, and the messages are transcribed from the agent and session validators:

- `Request body must be a JSON object.`
- `title must be a string.` / `title must be at most 200 characters.` / `title is required.`
- `body must be a string.` / `body must be at most 100000 bytes.` / `body is required.`
- `Unknown source "x".`

The two limits use different units on purpose. `title` is capped in UTF-16 code units, matching the rest of the API and the frontend's character counters. `body` is capped in **UTF-8 bytes** through `KNOWLEDGE_MAX_BODY_BYTES`, because that limit bounds storage and request size rather than what a user can read. Each message names its own unit. The default of 100,000 sits inside `MAX_BODY_BYTES`, so the body limiter never fires first and a client always gets the specific message rather than a generic 413.

A patch carrying no writable key returns the document unchanged and does not touch `updatedAt`, so an empty patch cannot reorder the list.

Search is scored by term overlap. Query words shorter than three characters are dropped, because they match almost every document and only flatten the ranking. The score is the fraction of remaining query words found in the title and body together, rounded to two places, and documents scoring zero are excluded. Ties break by most recently updated, so results are stable rather than dictionary ordered. Each hit carries an excerpt of at most 300 characters centred on the first matching term, falling back to the leading characters when the document matched on its title alone. Both storage backends call the same pure functions in `knowledge/search.py`, so they rank identically.

Four sample documents seed into an empty library on first boot, the way agents do. They are ordinary rows marked `source: "seed"` and can be deleted; a restart never brings one back.

2.3 Data model

2.3.1 Entities and relationships

The persistent domain is six tables: agents, chat sessions, triggers, runs, the tool calls belonging to each run, and knowledge documents. Agents, sessions, triggers, and documents are mutable operational resources. Runs and run tool calls are immutable history.

The split matters because the two kinds need opposite lifecycle rules. A user must be able to rename or delete current configuration without silently changing the evidence of what the system already did. At the same time, deleting a chat thread is expected to remove the turns that make up that thread.

Figure 9 shows the persistent entities with their deletion and retention rules. `knowledge_documents` stands alone in that picture because it really has no relationships: the library is global, so the table carries no `agent_id` and no foreign key.

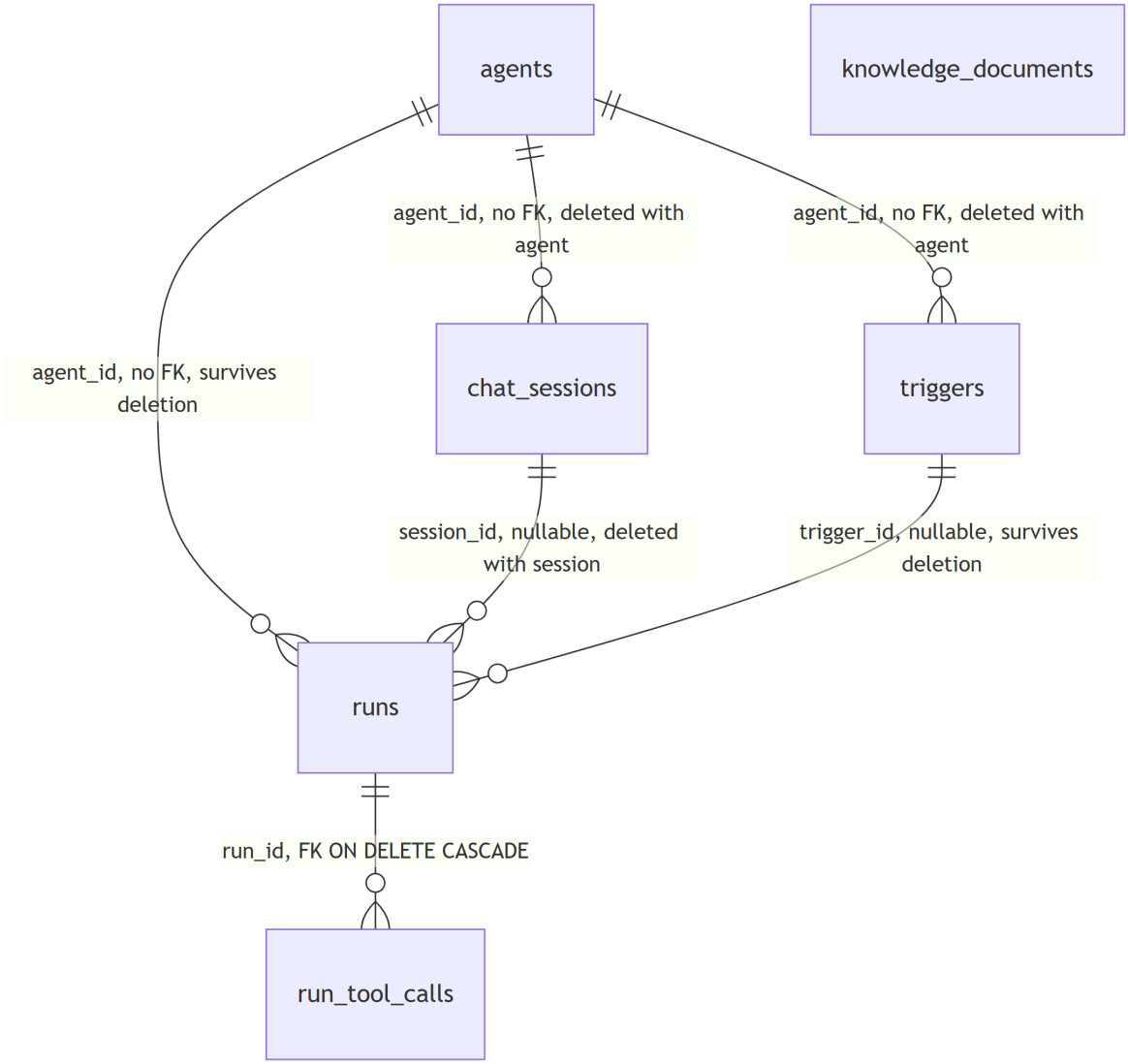

Figure 9: Persistent entities and relationships

Design decisions.

- **Runs snapshot `agent_name`, `model`, and `system_prompt`** because agents are mutable, and resolving those through the current agent would make an edit rewrite history.
- **No table has a foreign key to `agents`.** That is what allows the application to preserve runs while deleting the sessions and triggers that cannot function without the agent. A database-level cascade would make the audit trail impossible to keep.
- **`session_id` and `trigger_id` on `runs` are both nullable** and are what distinguish the two execution contexts. A chat run has a session and no trigger; a triggered run has the reverse.
- **`run_tool_calls.run_id` is a real cascading foreign key**, because a tool call has no meaning or lifecycle independent of its run.
- **PostgreSQL is the durable store; the in-memory store is a full peer.** Both satisfy the same repository protocols, including ordering and copy semantics, and one shared contract test suite runs against both. That is what keeps `uv run pytest` honest without a database.

2.3.2 Deletion behaviour

Operation	Deleted	Preserved	Why
Delete an agent	The agent, its chat sessions, its triggers	Its runs and their tool calls	History outlives mutable configuration. A session or trigger pointing at a missing agent can never be continued or fired, so leaving it would only put a permanently broken row in a list
Delete a trigger	The trigger	Runs it fired	Trigger activity is historical evidence, on the same terms as an agent's
Delete a session	The session and its runs	The agent and unrelated runs	The thread and its turns are one thing to a user, and an orphaned run is unreachable from the UI
Delete a run	The run and its tool calls	Everything else	Tool calls are owned by the run
Delete a knowledge document	The document	Every agent and run	The library is global and independent of agents, and a run's tool call already stored the text it retrieved

The first three rules live in the owning application service, not in the database, so the memory and PostgreSQL backends behave identically. Only the run-to-tool-call rule is a database cascade.

2.3.3 Physical schema

Table	Columns	Indexes and constraints
`agents`	`id` PK, `name`, `icon`, `description`, `model`, `system_prompt`, `tool_ids` JSONB, `status`, `created_at`, `updated_at`	`agents_updated_at_idx` on `updated_at DESC`
`runs`	`id` PK, `agent_id`, `agent_name`, `model`, `system_prompt`, `user_message`, `answer`, `status`, `error`, `latency_ms`, `session_id`, `trigger_id`, `created_at`	Indexes on `created_atDESC` and on agent, session, and trigger id each with `created_atDESC`; no FK to agents, sessions, or triggers
`run_tool_calls`	`id` PK, `run_id`, `seq`, `tool_id`, `args` JSONB, `result` JSONB, `error`, `duration_ms` DOUBLE PRECISION, `status`	`run_idREFERENCESruns(id)ONDELETECASCADE`; index on `(run_id,seq)`

Table	Columns	Indexes and constraints
chat_sessions	id PK, agent_id, title, created_at, updated_at	Index on updated_atDESC; no FK to agents
triggers	id PK, agent_id, name, message, kind, interval_minutes, time_of_day, weekdays JSONB, timezone, enabled, next_run_at, last_run_at, last_status, last_run_id, created_at, updated_at	Partial index on next_run_atWHEREenabled, plus indexes on agent_id and updated_atDESC; no FK to agents
knowledge_documents	id PK, title, body, source, created_at, updated_at	Index on updated_atDESC; no agent_id and no FK, because the library is global

Two column choices are worth calling out. The due index on `triggers` is **partial** because a disabled trigger has `next_run_at = NULL`, so the index the scheduler polls on every tick stays proportional to the enabled set rather than to the whole table. And `run_tool_calls.duration_ms` is `DOUBLE PRECISION` because an in-process tool answers in tens of microseconds, and integer truncation reported every one of them as `0 ms`, which is indistinguishable from a call that never ran.

JSONB is decoded to Python objects once, in the connection initialiser, so no caller has to remember to parse `tool_ids` out of a string.

2.3.4 Startup schema management and backfill

With `STORE_BACKEND=postgres`, the lifespan opens the pool, applies `app/core/schema.sql`, and seeds the four sample agents **only when the `agents` table is empty**, logging `postgres ready; seeded 4 agents` on first boot and `seeded 0 agents` afterwards. The guard is what stops a restart from bringing back an agent someone deleted.

The four sample knowledge documents are seeded straight afterwards under the same guard, logging `seeded 4 knowledge documents` and then `seeded 0 knowledge documents`. With `STORE_BACKEND =memory` both seeds happen in the repository constructor instead, which the container caches once per process – the memory equivalent of "only when the table is empty".

Every statement in the schema file is idempotent. Two later changes could not be expressed with `CREATE TABLE IF NOT EXISTS` alone and have their own guarded statements: `runs.trigger_id` is added with `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`, and the `duration_ms` widening runs inside a `DO` block that checks `information_schema` first.

This is startup schema management, not a migration framework. It handles creation and the two recorded widenings. Any further change against a populated database is a manual job. Alembic is the intended answer once the schema changes under real data.

The lifespan then runs `backfill_sessions`, a one-time migration for run history written before sessions existed. It reads runs whose `session_id` is still null, oldest first, groups them by agent, creates one session per agent titled by the same deterministic truncation, and assigns the runs to it. It is a no-op on every boot after the first and on an empty memory store. A failure is logged and swallowed rather than blocking startup: booting with some history un-migrated is recoverable on the next boot, while failing to boot is a total outage.

2.4 Extension interfaces

The two extension points are narrower than the application on purpose. Adding a tool or a model provider should not require touching execution orchestration, HTTP routes, agent management, or persistence.

2.4.1 Tool boundary

A tool is one file holding its identity, its schema, and its behaviour. The registry owns the catalog; nothing else in the codebase knows which tools exist.

```
class ToolParam(WireModel):
    name: str
    type: Literal["string", "number", "boolean"]
    required: bool = False
    description: str


class ToolSchema(WireModel):
    id: str
    label: str
    description: str
    params: list[ToolParam]


class ToolResult(WireModel):
    ok: bool
    value: Any | None = None
    error: str | None = None


@runtime_checkable
class Tool(Protocol):
    schema: ClassVar[ToolSchema]

    async def execute(self, **kwargs: Any) -> ToolResult: ...
```

`ToolRegistry` exposes exactly what the rest of the system needs:

Method	Purpose
`list()`	Schemas for `GET/api/tools`, in registration order
`get(tool_id)`	One tool, or `None`
`resolve(ids)`	Stored ids into tools in the requested order; unknown ids raise a 400
`known_ids()`	The allowed id set, so agent validation never imports the catalog
`invoke(tool_id,**kwargs)`	Registry-level invocation entry point

Design decisions.

- **Expected failures are values, not exceptions.** A bad argument or a refused connection returns `ToolResult(ok=False, error=...)`, so the model sees the failure and explains it instead of the turn collapsing. Only a genuine infrastructure fault should propagate, and even then the ADK adapter catches it, records it as a failed call, and returns the error to the model.
- **`known_ids()` exists so `agents/` can validate without importing `tools.impls`.** That single method is what keeps the agent module free of the catalog, and `tests/test_structure.py` enforces it.
- **A tool that needs storage receives it as a constructor argument.** `default_tools( http_timeout_ms, knowledge)` is handed a `KnowledgeRepository` by `app/container.py` and passes it to `KnowledgeSearchTool`. A tool never imports a repository implementation, so the same tool works over the memory store and over PostgreSQL.
- **The ADK bridge builds a real signature.** The protocol is `execute(**kwargs)`, which advertises no parameters. Handed to ADK unchanged, every tool would look argument-less and the model would never pass one. `adk_adapter.py` builds an `inspect.Signature` from

`ToolSchema`, writes the docstring ADK turns into the function declaration, times each call to hundredths of a millisecond, and claims each call's position in the trace **before** awaiting it, so concurrent calls are ordered by when the model asked rather than by which finished first.

Built-in tools, in registration order:

id	Behaviour	Notable guard
`current_time`	Current time in an optional IANA zone, defaulting to UTC	An unknown zone is a failed call, not a 500
`http_request`	Fetches a URL and returns `{status, latencyMs,body}`	See below
`calculator`	Evaluates an arithmetic expression	Parses to an AST and walks it, because `eval()` on a model-supplied string is remote code execution. Exponents are bounded, and non-finite results are rejected before they can reach JSON
`knowledge_search`	Searches the knowledge library through `KnowledgeRepository.search`, scoring by term overlap	Reads the same documents the Knowledge page manages, so an uploaded file is searchable at once. A bad or empty query is a failed call, not a 500

`http_request` is the one tool reachable by prompt injection, because the model chooses the URL from user text. It allows only `http` and `https` and only `GET` and `HEAD`, resolves the hostname and requires **every** resolved address to be publicly routable (using `is_global`, rather than a list of negative flags that would let carrier-grade NAT ranges through), disables redirects, bounds the request by `TOOL_HTTP_TIMEOUT_MS`, and reads the body in chunks so it hangs up at 64 KB rather than buffering a large file or a compression bomb into the process. The remaining gap is recorded rather than hidden: the address is resolved and checked, then httpx resolves it again to connect, so a DNS record changing between the two could still slip through. Closing that needs connection-level pinning, which is beyond this proof of concept.

2.4.2 LLM provider boundary

The provider port represents one complete agent turn, including any tool loop the model drives. It is not a single text completion.

```
@dataclass
class RunSpec:
    agent_id: str
    name: str
    model: str
    system_prompt: str
    tools: list[Tool]
    user_message: str
    retry: bool
    session_id: str | None = None
    trigger_id: str | None = None
    timezone: str | None = None


RunEvent = ToolCallStarted | ToolCallFinished | TextDelta | TurnFinished


class LLMProvider(Protocol):
    def models(self) -> list[ModelInfo]: ...
```

```
    def run(self, spec: RunSpec) -> AsyncIterator[RunEvent]: ...
```

Event	Data	Meaning
`ToolCallStarted`	`call_id`, `tool_id`, `args`	A tool invocation began
`ToolCallFinished`	`call_id`, `result`, `error`, `duration_ms`	That invocation completed or failed
`TextDelta`	`text`	Incremental model text, forwarded by the streaming route
`TurnFinished`	`text`, `model`, `latency_ms`	The authoritative final result; exactly one per turn

Design decisions.

- **Normalising here is what makes the mock and the live provider interchangeable**, and what lets the whole contract suite run without an API key. The cost is that every adapter must preserve call pairing, invocation order, result and error shape, model identity, and latency while hiding whatever its SDK emits beyond those four events.
- `RunSpec.timezone` **exists because the container clock is UTC** but a trigger stores the zone it was configured in. The ADK adapter turns it into an instruction paragraph, so a time-aware tool call with no explicit argument defaults to the zone the user picked rather than to the container's.
- **The optional SDK import is deferred into the methods that use it.** A module-scope import would make `google-adk` mandatory just to boot or test the service, and `tests/test_structure.py` imports every module.

`AdkGeminiProvider` **specifics.** It drives ADK's `Runner` in SSE streaming mode and forwards text as `TextDelta`, dropping parts flagged `thought=True` so the model's internal reasoning never reaches the user. Tool events are replayed from the recorder **after** the runner completes rather than correlated with ADK's own call ids, because ADK does not report durations and `durationMs` is part of the contract. The agent's ADK name is its **id**, because ADK requires a Python identifier and `Support Bot` is not one; the display name reaches the model through an instruction paragraph appended after the system prompt, which also overrides ADK's own identity line. Any SDK exception becomes a `ProviderError` carrying a classified sentence.

2.5 Deployment guide

2.5.1 Run the container stack

Prerequisite: Docker Desktop or another Docker Compose-compatible engine. This path needs no local Python and no local Node.

1. Move to the deployment directory:

```
Set-Location .\AgentPlatform-BackEnd\deployment
```

2. Build and start:

```
docker compose up --build
```

A fresh clone needs no configuration file: every setting has a default in `docker-compose.yml`, so the stack comes up against PostgreSQL with the deterministic mock provider, no credentials, and no password gate. Open `http://localhost:8080`.

3. Confirm what should have happened:

- `db` becomes healthy once `pg_isready` succeeds.
- `api` starts only after that, applies `app/core/schema.sql`, and listens on 4000 inside the network. It publishes no host port, so the only way in is through nginx.
- `web` starts only after `GET /api/health` succeeds, serves the built client, and proxies `/api/` to `api:4000` on the same origin.
- First boot logs `postgres ready; seeded 4 agents` and `seeded 4 knowledge documents`; later boots log `seeded 0` for both.

To change anything, copy the template and edit it:

```
Copy-Item .\.env.example .\.env
```

Two lines in that template change behaviour as soon as it is copied. `WEB_PORT=8090` moves the UI to `http://localhost:8090`, and `WEB_PASSWORD=change-me-before-going-public` turns the login gate on, so the first page load asks for a password instead of showing the app. Clear or change both lines to taste. Compose reads `.env` from this directory automatically because it is the directory holding `docker-compose.yml`. The file holds credentials, is gitignored, and must never be committed.

The `pgdata` named volume survives `docker compose down`. To discard the database and start empty:

```
docker compose down -v
```

That permanently deletes every agent, session, trigger, run, tool call, and knowledge document in the volume.

2.5.2 Configuration reference

`app/config.py` is the only module that reads the environment. It also reads a `.env` file in the server directory, which is worth knowing during local development: a stale local `.env` overrides the code defaults for anything run from that directory, including the test suite.

Variable	Code default	In the base Compose stack
`PORT`	`4000`	Declared but never read. The listening port comes from the container command, and from the `--port` flag locally
`CORS_ORIGIN`	`http://localhost:5173`	Forwarded from `.env`, defaulting to `http://localhost:8080`; a fallback only, since nginx makes the browser same-origin
`STORE_BACKEND`	`memory`	Fixed to `postgres`. It describes the topology, so setting it in `.env` does nothing
`DATABASE_URL`	`postgresql://app:app@db:5432/agents`	Built by Compose from `POSTGRES_PASSWORD`
`LLM_PROVIDER`	`mock`	Forwarded from `.env`; `mock` or `adk_gemini`
`GEMINI_API_KEY`	empty	Forwarded from `.env`
`TOOL_HTTP_TIMEOUT_MS`	`5000`	Forwarded from `.env`
`LOG_PAYLOAD_MAX_BYTES`	`32768`	Forwarded from `.env`
`KNOWLEDGE_MAX_BODY_BYTES`	`100000`	Forwarded from `.env`; UTF-8 bytes, and kept inside `MAX_BODY_BYTES`
`TRIGGERS_ENABLED`	`true`	Forwarded from `.env`
`TRIGGER_TICK_SECONDS`	`30`	Forwarded from `.env`
`TRIGGER_MAX_PER_TICK`	`20`	Forwarded from `.env`

Variable	Code default	In the base Compose stack
MAX_BODY_BYTES	262144	Code default; not exposed by the base Compose file

Only variables named in the `api.environment` block of `docker-compose.yml` reach the API container. There is no `env_file`, and that is intentional: the block is the complete, readable inventory of what the container receives, which is also how the Cloudflare tunnel token stays out of the API's environment. Adding a setting to `app/config.py` means adding a line there too. Putting a name in `.env` alone does nothing for an unlisted setting.

Compose itself, and the web container, also read:

Variable	Default	Scope
WEB_PORT	8080 in the compose file, 8090 in .env.example	Host port nginx is published on
WEB_BIND	0.0.0.0	Host interface that port binds to
WEB_PASSWORD	empty in the compose file, a placeholder in .env.example	Shared password for the nginx gate; empty leaves the site open
POSTGRES_PASSWORD	app	Database initialisation, and the generated DATABASE_URL
PUBLIC_ORIGIN	none	Required by docker-compose.prod.yml
CLOUDFLARE_TUNNEL_TOKEN	none	Required by docker-compose.prod.yml

PostgreSQL applies `POSTGRES_PASSWORD` at initdb only. Changing it against an existing `pgdata` volume has no effect, so set it before the first boot.

`WEB_PASSWORD` is rendered into an nginx `map` at container start, from `nginx-templates/gate.conf.template`, so the value never enters the image or the repository. An unset password renders an empty key, which is also what an absent cookie carries, so the map matches everyone and the gate does nothing. When it is set, every location except the login page and its stylesheet checks the cookie. The UI redirects to the login page and `/api/` answers 401 in the normal error envelope, so a `fetch` call gets a readable message instead of an HTML page. The compose file also sets `NGINX_ENVSUBST_FILTER` so the template's own nginx variables survive substitution.

2.5.3 Run against live Gemini

1. Set both values in `AgentPlatform-BackEnd/deployment/.env`:

```
LLM_PROVIDER=adk_gemini
GEMINI_API_KEY=your-key
```

2. Rebuild and restart the API, so the image carries the `adk` extra:

```
docker compose up -d --build api
```

3. Check that `GET /api/health` reports `"mode": "live"`, then send a chat message.

The health response reports configuration only. An empty or invalid key is discovered on the first provider call, which answers 502 `provider_error`.

The API has no authentication and no rate limiting of its own. Every route is open, including `DELETE /api/agents/{id}`. An exposed instance running `adk_gemini` with no gate is a free

LLM proxy on your key. Before publishing one, do all three of the following: set `WEB_PASSWORD` so nginx gates both the UI and `/api`, set `WEB_BIND=127.0.0.1` so the published host port is not an open door that bypasses the proxy, and put an access-control layer in front. The repository's production overlay uses a Cloudflare Tunnel with Cloudflare Access for that last part.

2.5.4 Common deployment problems

Symptom or task	Cause and action
The UI is not on 8080	You copied `.env.example`, which sets `WEB_PORT=8090`. Change the line or use that port
The site asks for a password after copying `.env`	`.env.example` ships a placeholder `WEB_PASSWORD`. Clear it to reopen the site, or use the value you set
Port already in use	Change `WEB_PORT` in `.env`
A changed database password has no effect	PostgreSQL kept the password stored in the existing `pgdata` volume. Migrate the credential, or recreate the volume if the data is disposable
Live mode starts but chat returns `provider_error`	Check `GEMINI_API_KEY`; startup never validates it
`STORE_BACKEND=memory` in `.env` is ignored	Intentional. The base file fixes it to `postgres`, because `memory` would detach the API from the database it just waited for
A knowledge document is refused as too large	`KNOWLEDGE_MAX_BODY_BYTES` is measured in UTF-8 bytes. Raise it and `MAX_BODY_BYTES` together, keeping the first below the second
Data vanished after a reset	`docker compose down -v` deletes the named volume; `docker compose down` alone preserves it

2.5.5 Run locally without containers

The two projects have separate toolchains and no root workspace. Run each command from the directory named.

Backend, from `AgentPlatform-BackEnd/server`, on Python 3.14 managed with `uv`:

```
uv sync
$env:STORE_BACKEND = "memory"
$env:LLM_PROVIDER = "mock"
uv run uvicorn app.main:app --port 4000 --reload
```

Memory and mock are already the code defaults. Setting them explicitly states the deterministic intent and overrides any stale local `.env`. **Port 4000 is load-bearing**: the Vite proxy, the nginx configuration, and the Compose healthcheck all target it.

Frontend, from `AgentPlatform-FrontEnd/client`, on Node 24:

```
npm ci
npm run dev
```

Vite listens on 5173 and proxies `/api` to `http://localhost:4000`, which is why `api-host.ts` resolves to a blank base in development: requests stay same-origin and no preflight happens.

2.6 Developer guide

2.6.1 Add a tool

1. Create a module under `app/modules/tools/impls/`, for example `weather.py`.
2. Implement `Tool`: declare `schema: ClassVar[ToolSchema]`, implement `async def execute(self, **kwargs) -> ToolResult`, and return `ToolResult(ok=False, error=...)` for expected argument or business failures. Let genuine infrastructure faults propagate; the provider adapter records them as a failed call rather than a 500.
3. Register it in `default_tools()` in `app/modules/tools/registry.py`. Its position in that list is its position in `GET /api/tools` and in the frontend picker.
4. If it needs storage or any other collaborator, take it as a constructor argument and add it to the `default_tools()` signature so `app/container.py` supplies it. Never import a repository implementation from a tool. `knowledge_search` is the worked example.
5. If deterministic demonstrations should exercise it, add a fixture to `TOOL_FIXTURES` in `app/modules/llm/providers/mock.py`. Without one the mock skips the tool, which is a deliberate degradation rather than a break, while the live provider still receives it normally.
6. Add tests for schema identity and parameter shape, a successful execution, an expected failure as `ToolResult(ok=False, ...)`, and registry resolution if relevant.
7. Run:

```
uv run pytest tests/unit/test_tools.py tests/unit/test_tool_registry.py
uv run ruff check .
```

Expected result. The schema appears in `GET /api/tools`, agents accept its id in `toolIds`, and the live provider can call it, with no change to `ExecutionService`, any router, or agent validation.

2.6.2 Add an LLM provider

1. Create a module under `app/modules/llm/providers/`.
2. Implement `LLMProvider`: `models()` and `run(spec)`.
3. Translate the SDK's native events into only `ToolCallStarted`, `ToolCallFinished`, `TextDelta`, and `TurnFinished`. Preserve call pairing, invocation order, result and error shape, model identity, and latency, and emit exactly one `TurnFinished`.
4. Convert SDK and upstream failures into `ProviderError` so the API answers 502 rather than 500, and put the classification in `llm/failures.py` rather than interpolating vendor text into a message a user will read.
5. Keep an optional SDK import inside the method that uses it. A module-scope import would make the SDK mandatory to boot or test the service, and `tests/test_structure.py` imports everything.
6. Select it in `get_llm_provider()` in `app/container.py` and add its literal to `Settings.llm_provider` in `app/config.py`. Nowhere else.
7. If it changes the model catalog, update `app/modules/llm/catalog.py` and the frontend's `src/config/models.ts` together. The frontend keeps its own copy for labels, and the two must agree or an agent will show a raw model id.

8. Add tests for model listing, a successful event sequence, tool-call translation, streaming text, and a provider failure.

9. Run:

```
uv run pytest tests/unit
uv run ruff check .
```

Expected result. Switching `LLM_PROVIDER` changes model execution with no change to `ExecutionService`, the routes, agent validation, or run persistence.

2.6.3 Add a storage backend

Implement every repository `Protocol` that applies – `AgentRepository`, `RunRepository`, `SessionRepository`, `TriggerRepository`, and `KnowledgeRepository` – under the owning module's `repositories/` package, and select it only in `app/container.py`. Run the shared repository contract suite against the new implementation: ordering, copy semantics, search ranking, and the deletion rules in section 3.2 are part of the contract rather than of any one backend.

2.6.4 Verification

Backend, from `AgentPlatform-BackEnd/server`:

```
uv run pytest
uv run ruff check .
```

The suite needs no database. PostgreSQL-backed tests skip unless `TEST_DATABASE_URL` points at a **throwaway** database. It is `TEST_DATABASE_URL` and never `DATABASE_URL`, because the fixtures drop and truncate the tables, and keeping the two names apart is what stops a stray test run from erasing real history.

```
uv sync --extra adk
$env:TEST_DATABASE_URL = "postgresql://app:app@localhost:5432/agents"
uv run pytest
```

Frontend, from `AgentPlatform-FrontEnd/client`:

```
npm test
npm run typecheck
npm run lint
npm run build
```

The backend suite has four layers, which is also the order to reach for when something breaks:

Layer	What it pins
`tests/contract/`	Status codes, exact validation messages, camelCase wire shapes, and the mock's fixtures
`tests/unit/`	Validation order, the event translator, the scheduler's schedule maths, tool behaviour, provider adapters
`tests/repositories/`	One shared contract suite run against both the memory and PostgreSQL implementations
`tests/test_structure.py`	The import boundaries the design rests on

2.6.5 Extension checklist

Before treating either extension as complete:

1. Confirm `GET /api/tools` or `GET /api/models` exposes the new capability.
2. Create or update an agent that selects it.
3. Run a successful chat turn and inspect both the returned tool calls and the persisted run.
4. Exercise one expected failure and confirm it uses the documented error envelope or the failed tool-call shape, not a 500.
5. Run the full backend suite and the linter.

The extension is correctly isolated when no route, agent-management rule, or execution step needs provider-specific or tool-specific branching.

2.7 Known limitations

Stated plainly, because they are consequences of the decisions above rather than oversights.

- **No user model, and no rate limiting.** The API itself is entirely open. The only protection available is the shared nginx password described in section 5.2, which has no per-person identity, no logout, and no authorisation rules. This is the single largest gap before any real exposure.
- **A turn carries no conversation history.** `RunSpec` holds one `user_message`, `ExecutionService` only appends to the run repository and never reads from it, and `AdkGeminiProvider` builds a fresh session service per turn. A session therefore groups and replays runs for the *reader*; the model itself sees only the system prompt and the current message. Adding memory means loading the session's runs in `_spec()` and extending the port to accept them. The data is already stored and ordered.
- **Knowledge search does not scale past a small library.** Every query is a linear scan over whole documents, scored by term overlap, with no chunking and no embeddings. It suits tens to low hundreds of short documents. Replacing it is a change behind `KnowledgeRepository .search` that no caller would see.
- **The scheduler assumes one active instance.** The compare-and-swap claim prevents a double firing, but scheduling still runs inside the API process. Scaling horizontally means moving it to a singleton worker or a managed scheduler.
- **Seeding can race across replicas.** `seed_agents` and `seed_knowledge_documents` each read a count and insert in one transaction, but at READ COMMITTED two replicas starting against an empty database can both read zero, and the loser fails on a unique violation during startup. An advisory lock or `ON CONFLICT DO NOTHING` would close it.
- **Startup schema management is not migrations.** Any schema change against a populated database is currently a manual job.
- **No retention on `runs` and `run_tool_calls`.** They grow without bound, and there is no backup or restore path for the `pgdata` volume.
- **The `http_request` guard has a resolve-then-connect gap.** Documented in section 4.1; closing it needs connection-level address pinning.
- **Credentials are plain environment variables**, visible to `docker inspect` and to any process inside the container.
- **Client IPs are the proxy's.** nginx forwards `X-Forwarded-For`, but the API does not read it, so logs attribute every request to nginx or the tunnel.